МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №1

з дисципліни «Алгоритми та структури даних»

Виконав:

студент групи ІМ-о51

Машута О.І

Перевірив:

ас. Череватенко О.В.

Київ 2026

Постановка задачі

Дане натуральне число n. Знайти суму перших n членів ряду чисел, заданого рекурентною формулою. Розв'язати задачу трьома способами:

1) у програмі використати рекурсивну функцію, яка виконує обчислення і членів ряду, і суми на рекурсивному спуску;

2) у програмі використати рекурсивну функцію, яка виконує обчислення і членів ряду, і суми на рекурсивному поверненні;

3) у програмі використати рекурсивну функцію, яка виконує обчислення членів ряду на рекурсивному спуску, а обчислення суми на рекурсив ному поверненні.

При проєктуванні програм слід врахувати наступне:

1) програми повинні працювати коректно для довільного цілого додатного n включно з n = 1;

2) видимість змінних має обмежуватися тими ділянками, де вони потрібні; 19

3) функції повинні мати властивість модульності;

4) у кожному з трьох способів рекурсивна функція має бути одна (за по треби, можна такожвикористати додаткову функцію-обгортку (wrapper function));

5) у другому способі можна використати запис (struct) з двома полями (але в інших способах у цьому немає потреби і це вважатиметься на длишковим);

6) програми мають бути написані мовою програмування С.

Варіант № 9

$$F_1 = (x-1)/x; \quad F_{i+1} = F_i \cdot i \cdot (x-1)/(i \cdot x + x), \quad i > 0;$$

$$\sum_{i=1}^{n} F_i = \ln x, \quad x > 0.5.$$

Текст програми:

```
#include <stdio.h>
#include <math.h>

double rec_descent(double x, int i, int n, double current_term, double
current_sum) {
    if (i == n) {
        return current_sum;
    }
    double next_term = current_term * i * (x - 1.0) / (x * (i + 1.0));
    return rec_descent(x, i + 1, n, next_term, current_sum + next_term);
}

double sum_descent(double x, int n) {
    double start_term = (x - 1.0) / x;
    return rec_descent(x, 1, n, start_term, start_term);
}

typedef struct {
    double term;
    double sum;
} RecResult;

RecResult rec_return(double x, int i) {
    if (i == 1) {
        double start_term = (x - 1.0) / x;
        RecResult res = { start_term, start_term };
        return res;
    }

    RecResult prev = rec_return(x, i - 1);
    int prev_i = i - 1;

    double current_term = prev.term * prev_i * (x - 1.0) / (x * (prev_i +
1.0));
    double current_sum = prev.sum + current_term;
```

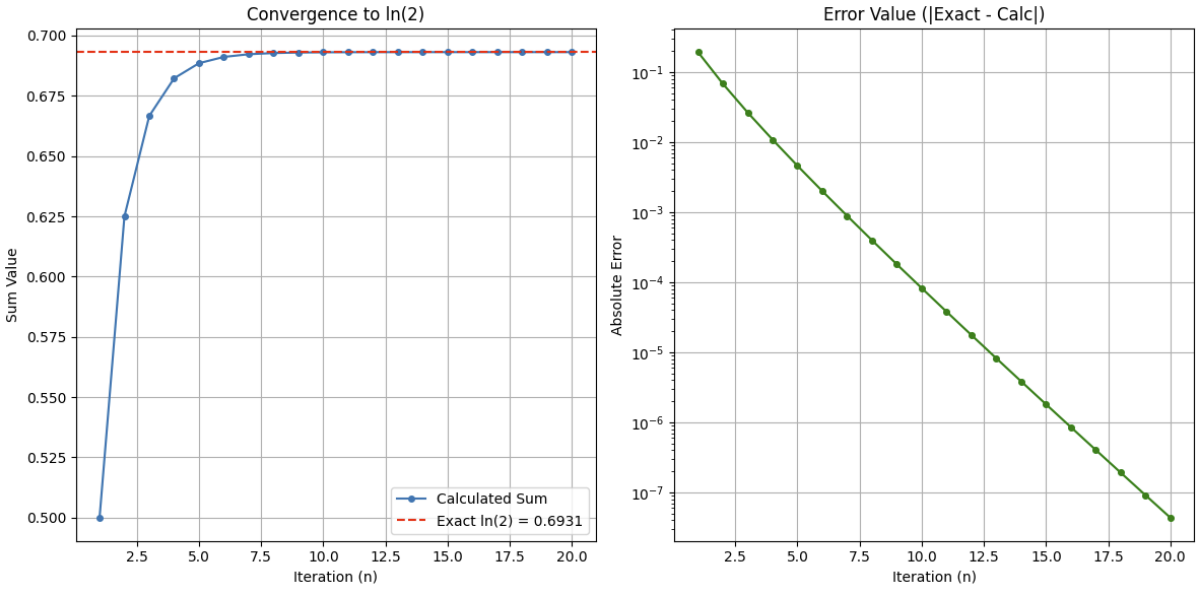
```
    RecResult current = { current_term, current_sum };
    return current;
}

double sum_return(double x, int n) {
    return rec_return(x, n).sum;
}

double rec_mixed(double x, int i, int n, double current_term) {
    if (i == n) {
        return current_term;
    }
    double next_term = current_term * i * (x - 1.0) / (x * (i + 1.0));
    return current_term + rec_mixed(x, i + 1, n, next_term);
}

double sum_mixed(double x, int n) {
    double start_term = (x - 1.0) / x;
    return rec_mixed(x, 1, n, start_term);
}

double sum_cyclic(double x, int n) {
    double term = (x - 1.0) / x;
    double sum = term;

    for (int i = 1; i < n; i++) {
        term = term * i * (x - 1.0) / (x * (i + 1.0));
        sum += term;
    }
    return sum;
}

int main() {
    int n_test = 5;
    double x_test = 2.0;

    printf("=== ALGORITHM TESTING (n = %d, x = %.2f) ===\n", n_test, x_test);

    double res_desc = sum_descent(x_test, n_test);
    double res_ret = sum_return(x_test, n_test);
    double res_mix = sum_mixed(x_test, n_test);
    double res_cyc = sum_cyclic(x_test, n_test);
    double res_exact = log(x_test);

    printf("1. Recursion (descent):       %.10f\n", res_desc);
    printf("2. Recursion (return):        %.10f\n", res_ret);
```

```
    printf("3. Recursion (mixed):         %.10f\n", res_mix);
    printf("4. Cyclic algorithm:          %.10f\n", res_cyc);
    printf("5. Reference math.h (ln x):   %.10f\n", res_exact);

    printf("\nError: |%.10f - %.10f| = %.10f\n", res_cyc, res_exact,
fabs(res_cyc - res_exact));

    return 0;
}
```

Тест програми:

```
=== ALGORITHM TESTING (n = 5, x = 2.00) ===
1. Recursion (descent):       0.6885416667
2. Recursion (return):        0.6885416667
3. Recursion (mixed):         0.6885416667
4. Cyclic algorithm:          0.6885416667
5. Reference math.h (ln x):   0.6931471806

Error: |0.6885416667 - 0.6931471806| = 0.0046055139
```

Графік для програми:



Висновок: Під час виконання лабораторної роботи №1 я засвоїв теоретичний матеріал та здобув практичні навички проєктування і створення рекурсивних алгоритмів мовою С.